MOCA template lifecycle automation

Self-serve sandbox templates, gated by policy, with provenance

sbx-template · 2026-11-05

Outline

6 sections; the numbers are slide numbers

1 Why and what

MOCA's motif, the problem, the design in four ideas

3 The brain thinks, the hands act; a template shapes the hands
4 Benchmarks need many templates; adding one was manual
5 Self-serve: any enrolled client builds; policy, not a person, decides

2 Architecture

where each part runs, and a build's states

6 One control plane decides; hosts build and run; clients only ask
7 A build has two states: how far from usable, and whether it runs now

3 Onboarding

create to registered, and the refusals

8 Create to registered: no approver, every decision recorded
9 Refusals are answers with a rule, not silence

4 Using a template

activation, first use, sessions, one control plane

10 Activation is a lease; the worker starts in seconds
11 First use: name a template and go
12 A Moca Pi session runs only on its build
13 One control plane serves both kinds of brain

5 Provenance and validation

records chained by digest; windows 1-4

14 Every step is a chained record, checkable offline
15 Four windows validated the lifecycle end to end
16 Activation fell from ~100 s to ~11-17 s
17 What the windows found, and the fixes

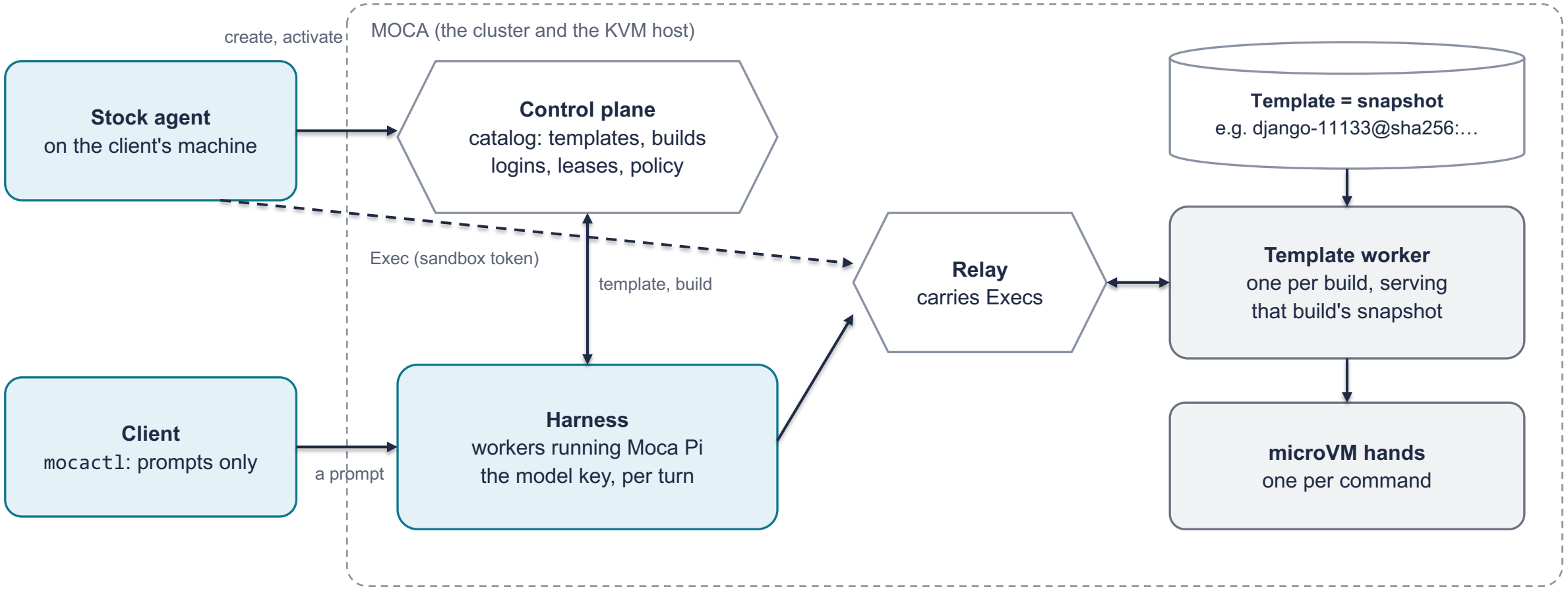
6 Next

deploy, model, upstream

18 Next: deploy the combined plane, then a real model, then upstream

The brain thinks, the hands act; a template shapes the hands

Moca Pi brain, or a stock agent, with MOCA microVM sandbox hands

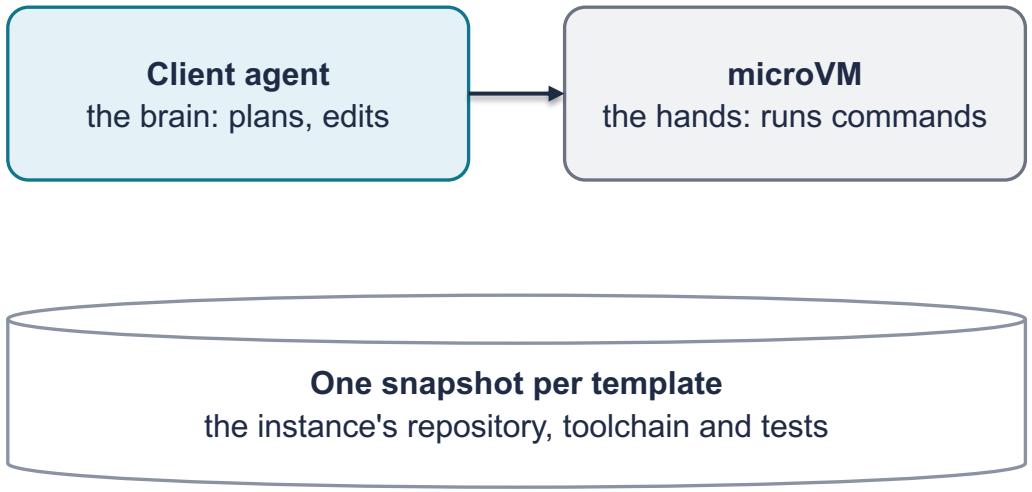


The hands never hold the model key or the agent
they get commands and return output; a template decides what the hands contain

Benchmarks need many templates; adding one was manual

Where the work started (spec §1-§2)

- **One template = one SWE-bench instance's environment**, as a microVM snapshot that every command of that instance runs in.
- **Before:** each template was a hand-run build, a hand-picked worker id, a settings edit and a redeploy of the control plane.
- **At scale:** 30+ per-instance templates, built and retired as experiments run, by clients that are not operators.
- **And proof:** which image, which checks, which snapshot served a given run.



The goal (the user, 2026-11-02)
SWE-bench at scale on templates, self-serve and gated, validated end to end before any upstream PR

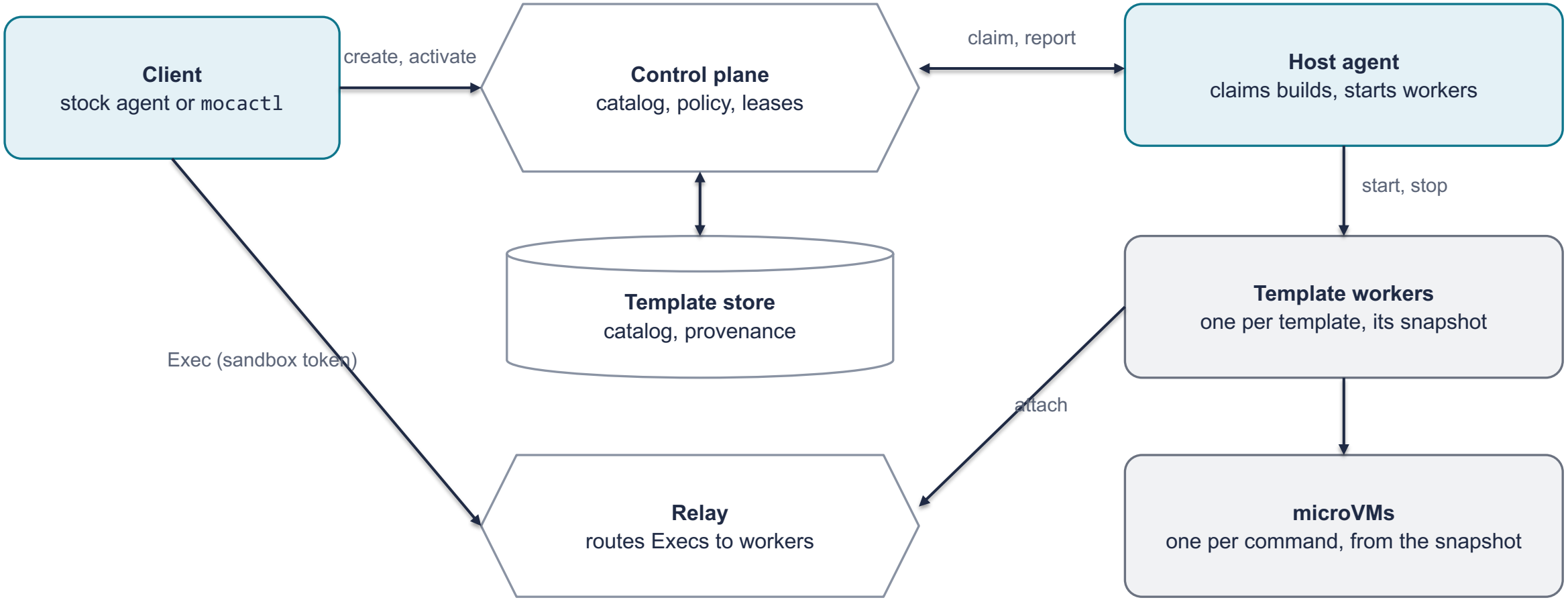
Self-serve: any enrolled client builds; policy, not a person, decides

The design in four ideas (spec §3, §6-§8; D12-D13)

Templates are declared in a store	a template is a name over immutable builds; a build's id is its snapshot's manifest hash
Policy gates every image	machine-checked rules at create and at registration; a `benchmark` profile needs a trusted gate record
Hosts build and run; the control plane decides	a host agent claims builds and starts workers; clients activate by lease, within quotas
Every step is a chained record	append-only provenance named by digest, verifiable offline with no access to the cluster

One control plane decides; hosts build and run; clients only ask

Who runs where: the cluster (OpenShift; e.g. ykt3) and the KVM host (bare metal; e.g. bm1)



The cluster holds no VM; the KVM host holds no policy. The host agent reads its wanted state from the control plane and reports facts back; readiness is what the relay sees attached.

A build has two states: how far from usable, and whether it runs now

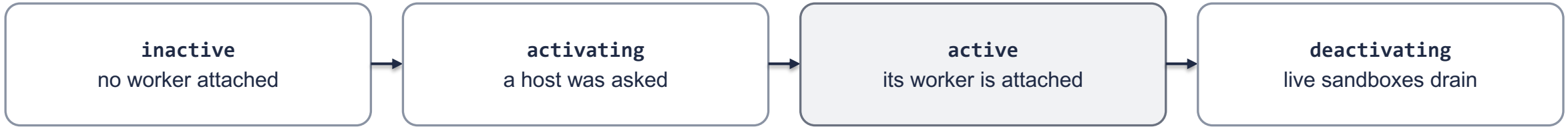
Onboarding is set by the host's reports and policy; readiness by what is attached

Onboarding (spec §3.3)



Off the path: `failed` (a build or probe failure, with its step; or the owner's cancel), `rejected` (policy at registration, with its rule), `retired`.

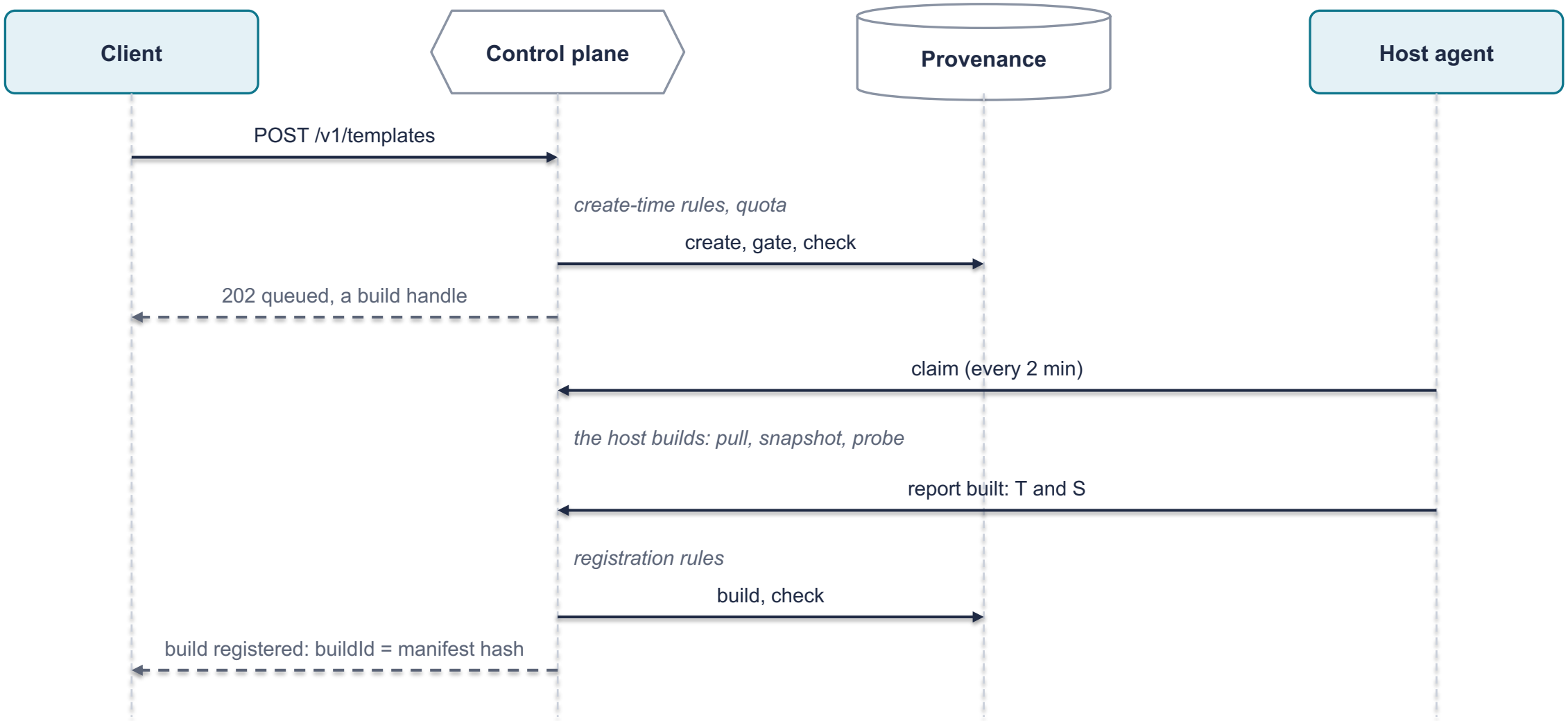
Readiness of a registered build



`unavailable`: no host holds its snapshot, or every holder is full. `active` is derived from the relay's attached record, so the state cannot claim a worker that is not there.

Create to registered: no approver, every decision recorded

Sequence 1: onboarding (spec §4, §6.1, §8)



Refusals are answers with a rule, not silence

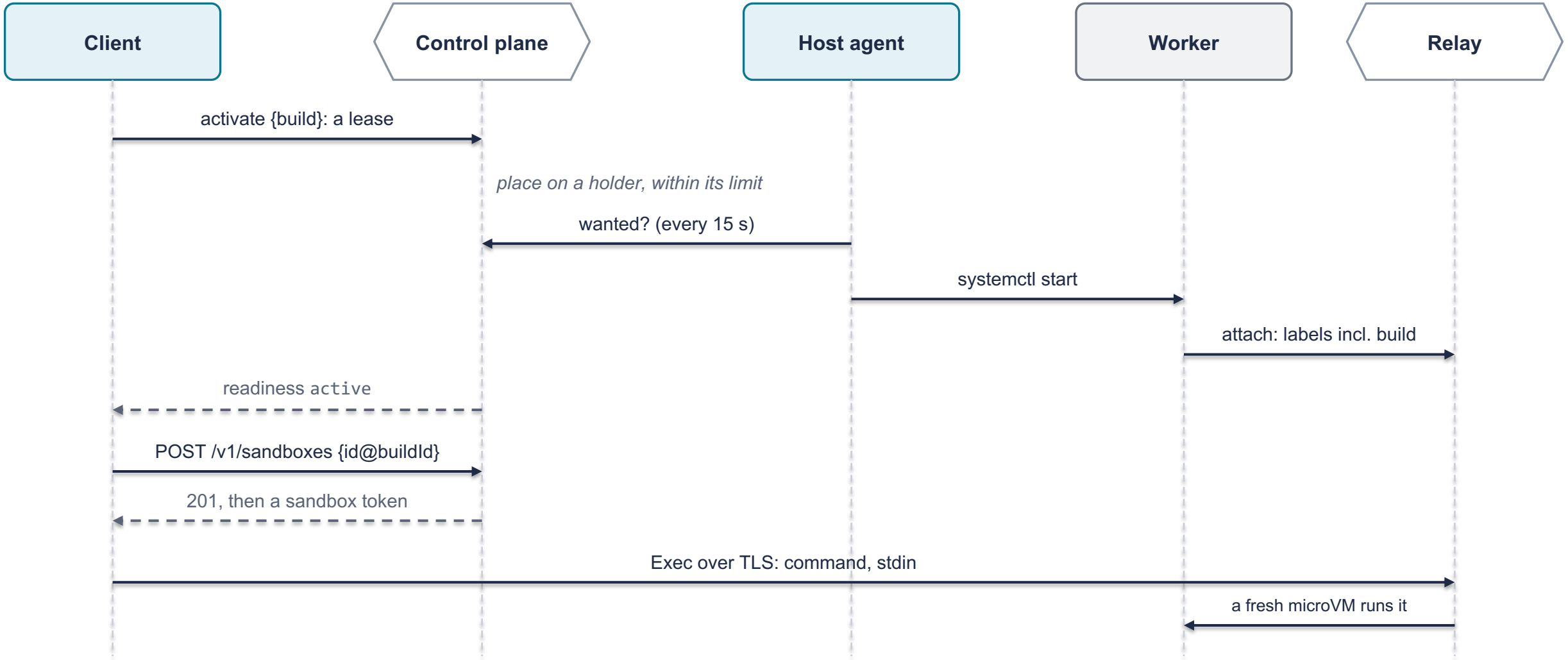
What a client sees when a create cannot proceed (spec §4, §8; G21, G22, G29)

Case	Answer	Recorded
Policy refuses the create	422 `policy_refused` with the rule id	a `check` record
Another subject owns the id	409 `template_exists`	nothing queued
Same inputs as a build a host still holds	200 or 202 with `reused: true`	no second build
Over a quota	429 `quota_exceeded`, naming which	nothing queued
The owner withdraws a build	cancel: `failed` at step `cancelled`	a `build` record
The build or probe fails on the host	`failed`, with step and code (`probe_failed`)	a `build` record

Nothing that names where an image lives leaves the host
failure text is redacted; a record naming the registry is not sent at all (G25, G28)

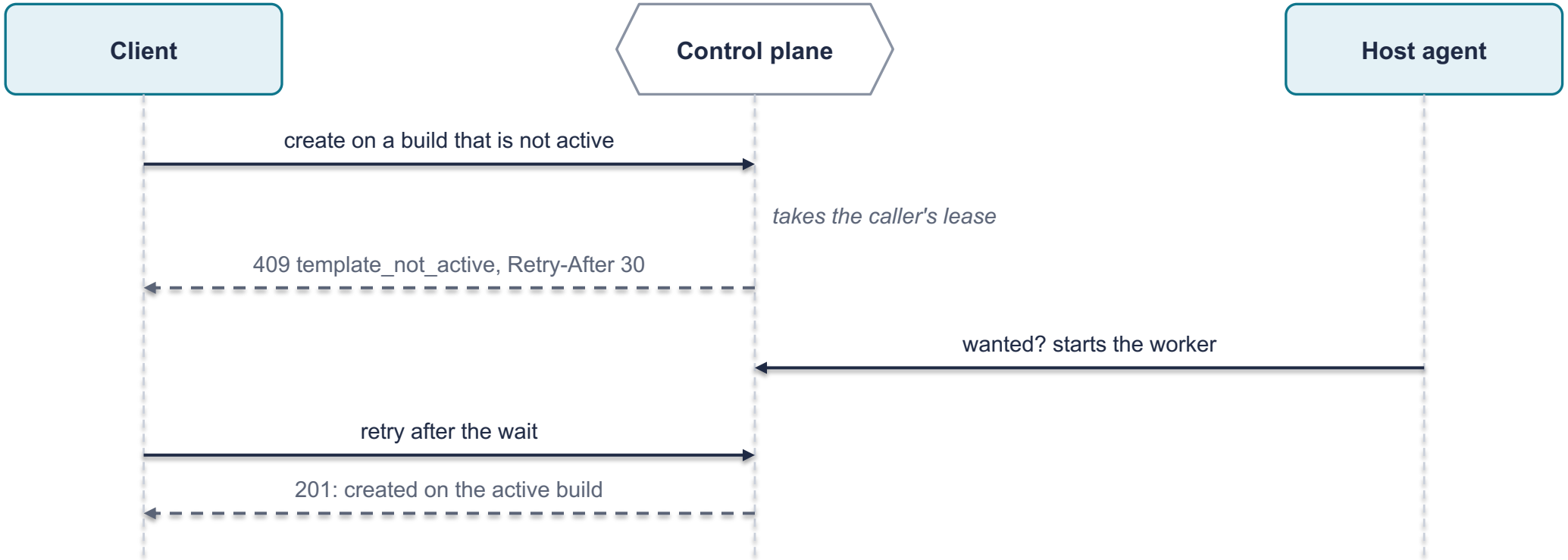
Activation is a lease; the worker starts in seconds

Sequence 2: activate, then a sandbox on the pinned build (spec §6.3; G31)



First use: name a template and go

Activation on first use (spec §6.4), for sandboxes and sessions alike

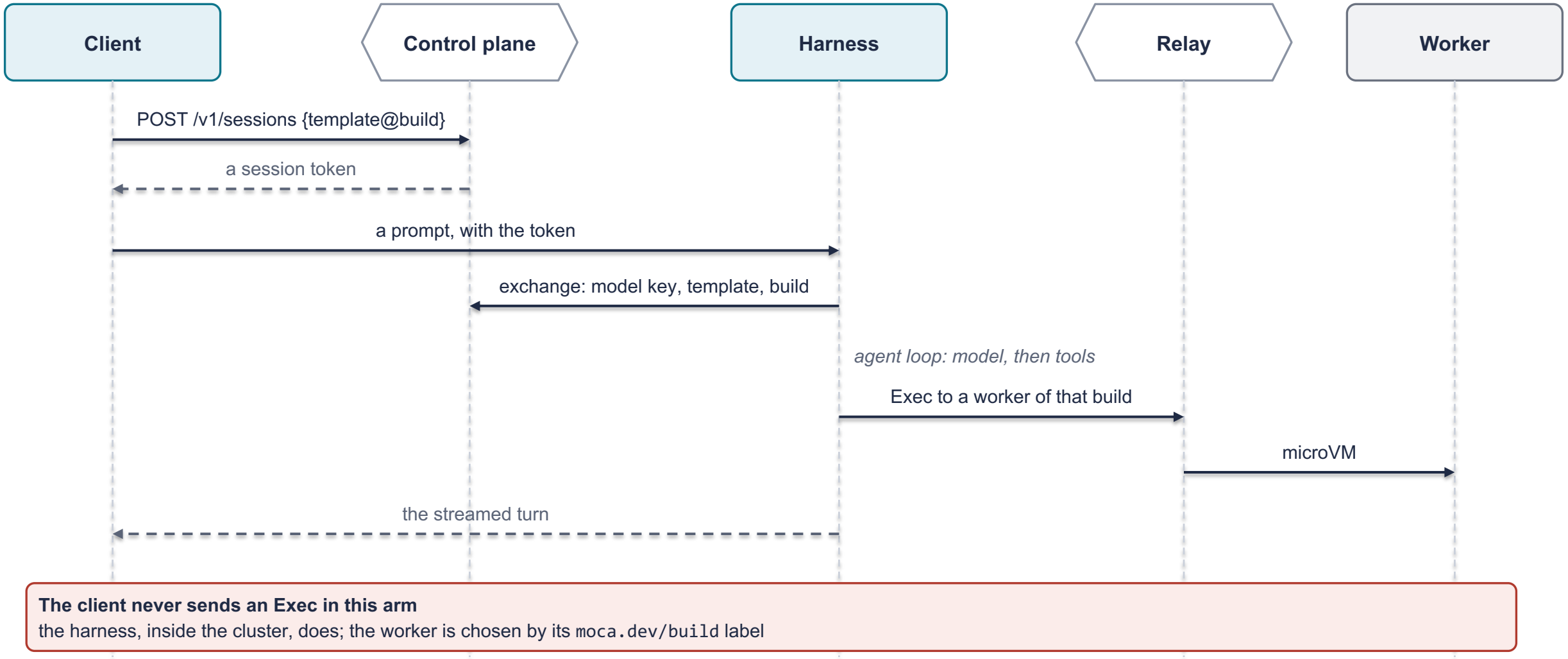


`activate: false` starts nothing. No sandbox is kept while waiting, so a wait never uses the caller's sandbox cap. `mocactl` retries for up to 10 minutes, saying why.

Without host agents nothing changes: the create succeeds as before

A Moca Pi session runs only on its build

Sequence 3: a session pinned to a build (combined control plane §3.4; G19)



One control plane serves both kinds of brain

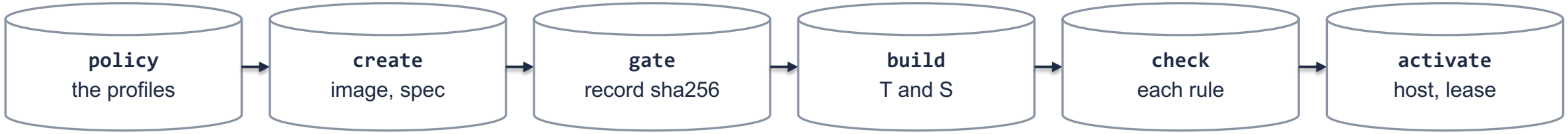
The combined route profile (combined control plane design, C1-C4)



Decision	Chosen
C1 How	a third profile, `combined`; `harness` and `sandbox` unchanged
C2 Login	open as on `harness`; enrollment checked on sandbox routes and template writes
C3 Where	a new namespace, so the validated SBX0 stays as it is
C4 Turns by build	in: a worker label and a filter (G19)

Every step is a chained record, checkable offline

Provenance (spec §7): digests, not signatures



- **One file per record**, named by the sha256 of its canonical JSON; each names its predecessor in `prev`, with a `seq`. Any edit, drop or reorder breaks the chain.
- **Stored in the control plane** on a volume, not Redis: records, a head per stream, and the cited gate records byte for byte.
- **Verified offline** by a stdlib Python script or the same rules in TypeScript, from an export, on a machine with no access to the cluster.

An export proves itself consistent, not complete
compare each stream's head with a copy kept elsewhere to catch a dropped tail

Four windows validated the lifecycle end to end

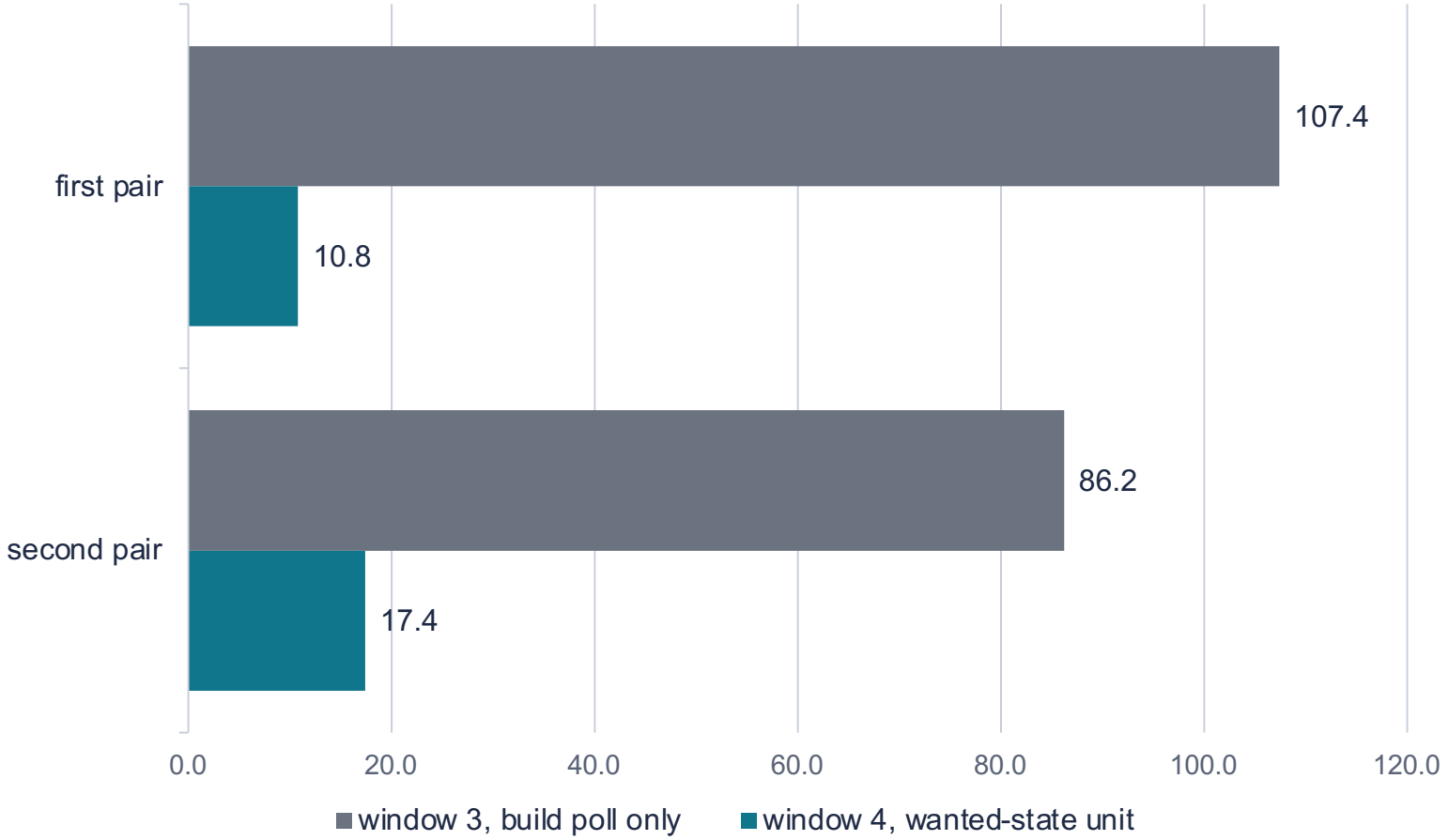
Spec §10 steps, as run 2026-11-04 to 2026-11-05 (no model)

Step	Window 1	Window 2	Windows 3-4
Create, build, register	4 of 4	4 of 4, fresh	4 of 4 each
Refused create (X10)	yes	yes	yes
Broken probe	not reached	`probe_failed`	step `probe` (w4)
First use	not built	409, then active	as w2
Served build = pinned	4 of 4	4 of 4	4 of 4
Patches match recordings	not compared	4 of 4	4 of 4
Offline verification	83 records	140 records, 12/12	not repeated

Step 3 (a second client on the same instance) passed separately with a second enrolled subject: its create and its new build `409`, its cancel `403`.

Activation fell from ~100 s to ~11-17 s

Seconds from each activation's request to `active`, both builds of a pair together (G31)



Before: an activation waited for the build poll, which starts 2 minutes after the previous one ends, and behind any build.

After: a separate unit applies the wanted state every 15 s; a lock keeps it and the build poll from acting at once.

One pair per window
a measurement, not a distribution

What the windows found, and the fixes

Every finding has a register entry; MOCA's are fixed and validated

Id	Found	Fixed in	Validated
G26	commands over 32 KiB hung the worker	the worker: refused at once	w2-w4
G25, G28	a registry host left the host in a reason	host agent redaction	w2-w4
G21, G29	duplicate builds; reuse of a removed snapshot	control plane reuse rules	w2-w4
G31	activation waited for the build poll	the wanted-state unit	w4
G19	a session could run on another build	the build label and filter	label w4
G30	~20 s client stalls	the client's DNS (not MOCA)	w4: 0 stalls

Not yet covered
a real model and grading (step 8); the combined control plane deployed (built, tested, not on a cluster)

Next: deploy the combined plane, then a real model, then upstream

In order, each with the user's decision first

1	**Deploy the combined control plane**	a new namespace on the cluster (a cluster-scoped change), a new image, a window on the KVM host with both brains
2	**A real model and grading**	spec §10 step 8, with the model spend agreed
3	**Propose upstream**	as PRs, from what the windows proved; upstream has no session templates today

Built, not deployed
the combined control plane's increments 1-5 are tested and pushed, not run on a cluster